

Αναφορά πάνω στην 1η εργασία

#####

ΑΕΜ: ####

25/10/2024

- **Περίληψη:**

Στην αναφορά παρακάτω παρουσιάζεται η πορεία μου στον προγραμματισμό ενός οδηγού ένδειξεων 7-τμημάτων της πλακέτας Nexys A-7 100T καθώς και λεπτομέρειες όσον αφορά την υλοποίηση μου, την επαλήθευση και την δοκιμή αυτής, καθώς και προβλήματα που αντιμετώπισα στην εκτέλεση των παραπάνω. Αυτό περιλαμβάνει τον προγραμματισμό και τη δοκιμή ενός αποκωδικοποιητή 7-τμημάτων, μιας μονάδας συγχρονισμού και anti-bounce για τη σωστή λειτουργία των εισόδων-κουμπιών, έναν οδηγό τεσσάρων ανόδων και διαφόρων μηχανισμών εμφάνισης ενός μυνήματος στις ενδείξεις

- **Εισαγωγή:**

Η εργασία είχε σαν κύριο στόχο την σωστή οδήγηση ένδειξεων επτά τμημάτων. Με τον διαχωρισμό της γίνεται ευκολότερη η διάκριση των επιμέρων στόχων της.

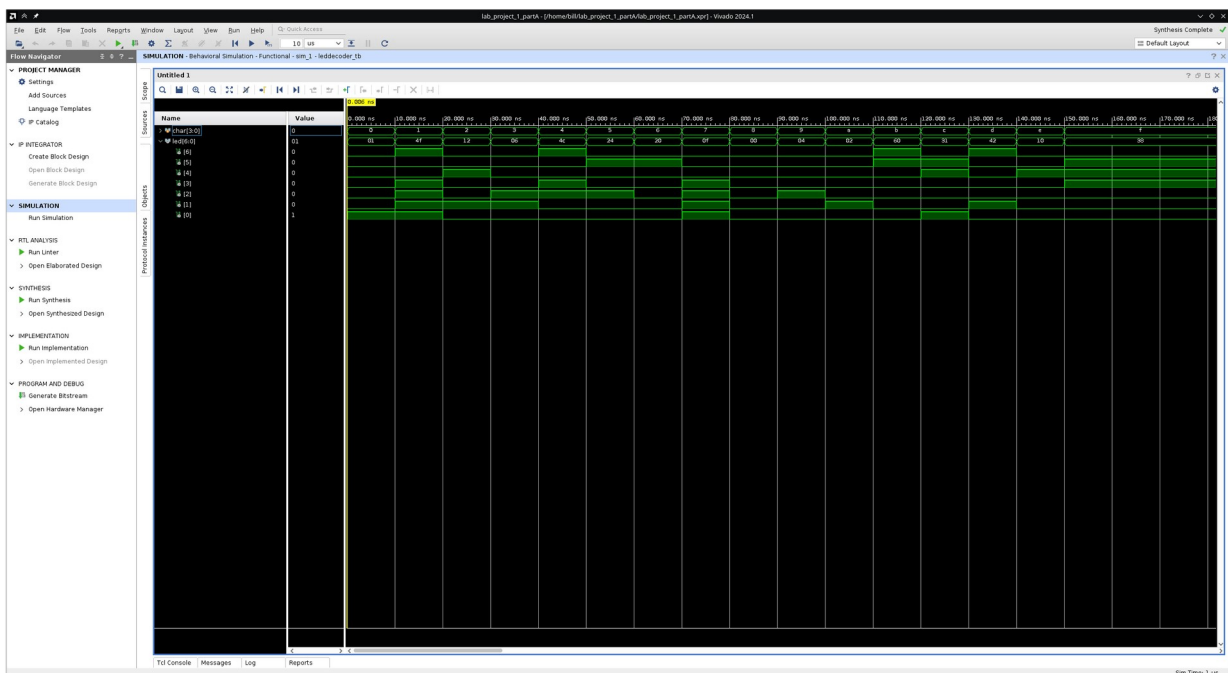
Πρώτο στόχο αποτελούσε ο προγραμματισμός αποκωδικοποιητή και η σωστή οδήγηση των καθόδων των ενδείξεων, το οποίο επιτεύχθηκε εύκολα με τη χρήση ενός switch statement και μερικών αναθέσεων. Στόχος ήταν επίσης η σωστή οδήγηση των ανόδων, κάτι που επιτεύχθηκε με τη χρήση μετρητή. Επιπλέον στόχο αποτέλεσε η αποτύπωση ενός μυνήματος με τη χρήση των ενδείξεων. Κρατώντας το μήνυμα σε καταχωρητή, χρειάστηκε τρόπος για τη “περιστροφή” του, την οποία υλοποιήσαμε με δύο τρόπους: αρχικά με τη χρήση ενός κουμπού και έπειτα με βηματική περιστροφή, χρησιμοποιώντας μετρητή.

- **Μέρος 1ο – Υλοποίηση αποκωδικοποιητή 7-τμημάτων:**

Το κύκλωμα που φτιάχτηκε στο πρώτο κομμάτι της εργασίας είναι ένας απλός αποκωδικοποιητής. Σάν είσοδο του έχει έναν 4-bit αριθμό, ο οποίος περιγράφει ποιο δεκαεξαδικό ψηφίο θέλουμε να εμφανιστεί στην ένδειξη των 7 τμημάτων, και σαν έξοδο του έχει έναν 7-bit αριθμό που υποδεικνύει ποιά στοιχεία της ένδειξης θα είναι ανοιχτά για να εμφανιστεί σωστά ο αριθμός, η καλύτερα, ποιά σήματα καθόδων θα οδηγούνται στο 0.

Η υλοποίηση του παραπάνω κυκλώματος συνδιαστικής λογικής μπορεί να επιτευχθεί με τη χρήση μίας μονάδας, η οποία περιέχει ένα switch case block, το οποίο περιέχεται μέσα σε ένα always block. Και τα δύο έχουν την είσοδο της μονάδας σαν μοναδικό στοιχείο της λίστας ευαισθησίας τους.

Για να ελέγξουμε την ορθή περιγραφή της μονάδας, αρκεί να ελέγξουμε οτι κάθε είσοδος αποκωδικοποιείται στη σωστή έξοδο. Το testbench ελέγχει το παραπάνω εισάγοντας κάθε είσοδο στη μονάδα, με διαφορά 10ns, επιτρέποντάς μας να ελέγξουμε την έξοδο διαμέσω των κυματομορφών. Παρακάτω φαίνεται ένα στιγμιότυπο που αποτυπώνει τις κυματομορφές αυτές, μέσω behavioural simulation στο vivado.

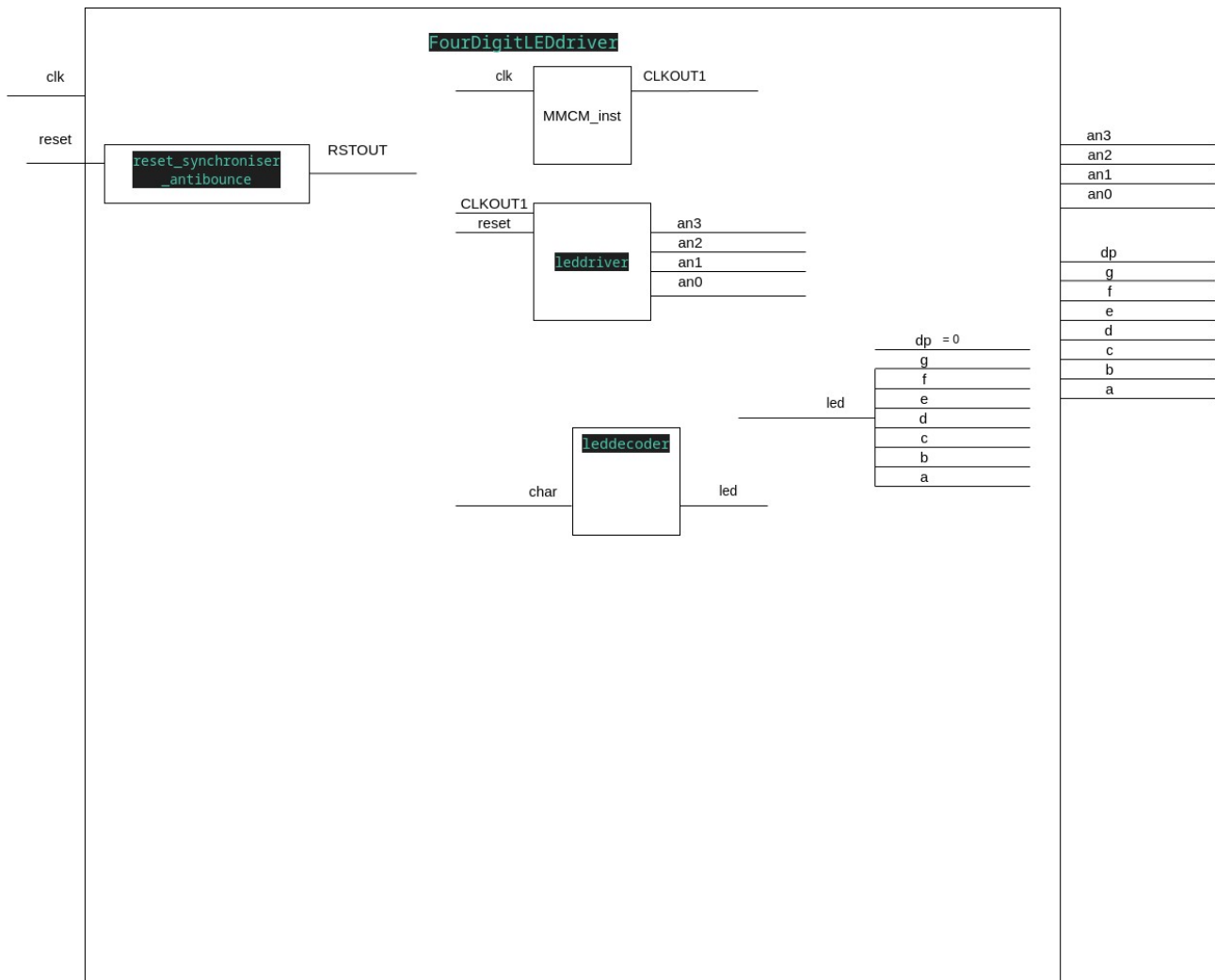


Part A Top Module Behavioural simulation

Αν και δεν μπορεί να χρησιμοποιηθεί το FPGA για τον έλεγχο αυτού του κομματιού της εργασίας, μπορούμε να διαβεβαιώσουμε οτι η σύνθεση και η υλοποίηση εκτελούνται επιτυχώς. Πράγματι, δεν προέκυψε κάποιο πρόβλημα κατα την σύνθεση ή την υλοποίηση.

- **Μέρος 2ο – Οδήγηση τεσσάρων ψηφίων:**

Σε αυτό το κομμάτι της εργασίας, θα χρειαστεί να προσθέσουμε τέσσερις επιπλέον μονάδες στο κύκλωμα: την μονάδα MMCM που θα μας επιτρέψει να έχουμε ρολόι με διαφορετική περίοδο απο το αρχικό, την μονάδα υπεύθυνη για την οδήγηση των ανόδων, μία μονάδα που θα λειτουργεί ως συγχρονιστής και ως anti-bounce για το σήμα reset ,καθώς το top module που θα είναι υπεύθυνο για το instantiation των υπόλοιπων μονάδων. Ένα διάγραμμα που αποτυπώνει τη ροή των δεδομένων φαίνεται παρακάτω



Part B Dataflow Diagramm

Η μονάδα leddriver είναι υπεύθυνη για τη σωστή οδήγηση των τεσσάρων ανόδων, το οποίο γίνεται με τη χρήση ενός 4-bit μετρητή και ένα switch case block. Αυτό επιτυγχάνεται με τη χρήση ενός always block με ασύγχρονο reset. Πρέπει τα σήματα των ανόδων να μην επικαλύπτονται όταν βρίσκονται στο 0 (δηλαδή είναι ενεργά), γιατί αυτό θα οδηγήσει στην αλλοίωση των ενδείξεων, οπότε κάθε άνοδος ανοίγει με διαφορά τεσσάρων κύκλων ρολογιού απο τη προηγούμενη. Για παράδειγμα, όταν το reset δεν είναι πατημένο, και η τιμή του counter, είναι 4'b0010, τότε η τιμή της πρώτης ανόδου θα οδηγηθεί στο 0, ενώ η τιμή όλων των υπολοίπων θα οδηγηθούν στο 1. Άρα στον επόμενο κύκλο θα οδηγηθεί και αυτή στο 0, και μετά απο 4 κύκλους η τιμή της δεύτερης ανόδου θα οδηγηθεί στο 0 με τη σειρά της

Η μονάδα anti-bounce είναι υπεύθυνη για τη σωστή καταγραφή μιας εισόδου, το οποίο περιλαμβάνει έναν συγχρονιστή και προφανώς μια μονάδα anti-bounce, έχοντας ένα always block για το καθένα από αυτά (και τα δύο έχουν στη λίστα ευαισθησίας τους μόνο το ρολόι). Υλοποιούμε τον συγχρονιστή με την χρήση δύο flip-flops, το οποίο στη verilog συνεπάγεται την ανάθεση της εισόδου σε ένα καταχωρητή, και την ανάθεση του καταχωρητή αυτού στην έξοδο, το οποίο θα γίνει μέσα στην always που αναφέρθηκε νωρίτερα. Το anti-bounce υλοποιείται με τη χρήση ενός 32-bit μετρητή, ο οποίος καθυστερεί την ανάθεση μέχρι ο μετρητής φτάσει σε μία συγκεκριμένη τιμή (εδώ μετά από 2147483648 κύκλους), η τιμή της εξόδου θα γίνει ίση με τη τιμή της εισόδου και ο μετρητής θα ξαναμηδενιστεί. Ο counter θα αυξάνεται μόνο αν οι καταχωρητές που έχουμε στα flip flops είναι ίσοι μεταξύ τους, αλλιώς θα μηδενίζεται.

Η μονάδα MMCM είναι ήδη υλοποιημένη, οπότε για να πετύχουμε την επιθυμητή συχνότητα των 5MHz αντί της αρχικής συχνότητας 100MHz, προσαρμόζουμε αντίστοιχα τις εισόδους της.

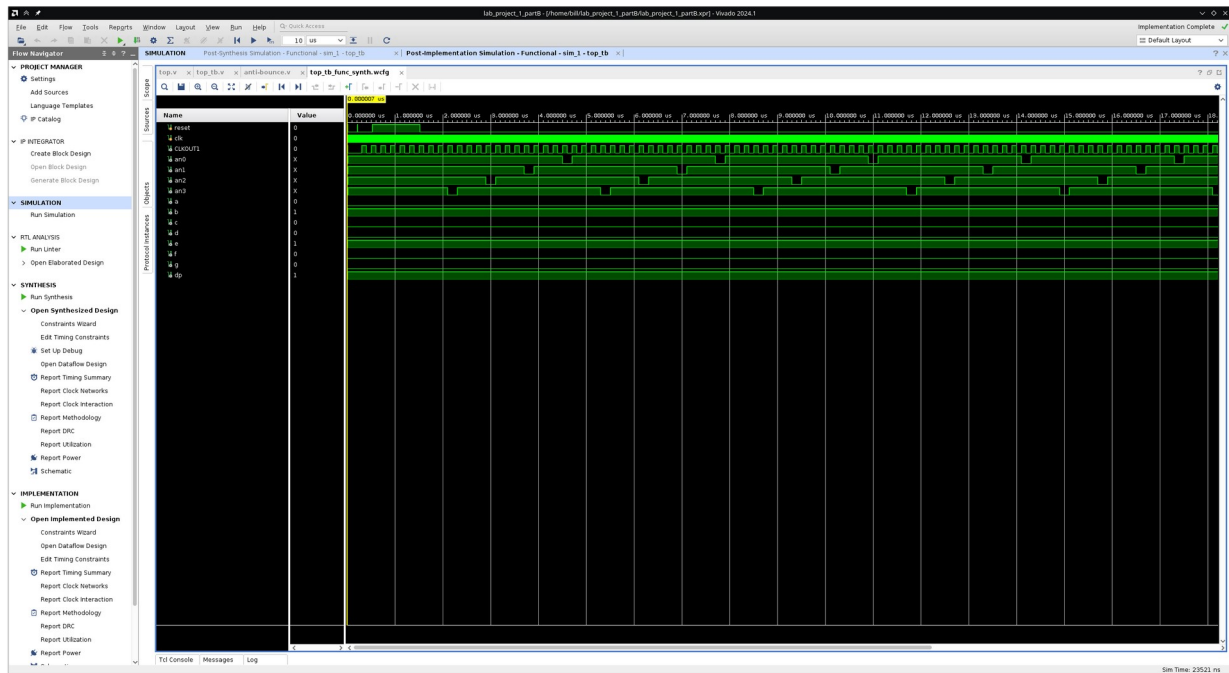
Η top μονάδα κάνει instantiate μία μονάδα MMCM (MMCME2_BASE_inst), η οποία μας δίνει ένα ρολόι CLKOUT1 με συχνότητα 5MHz (δηλαδή με 20 φορές μικρότερη συχνότητα από το αρχικό), μία μονάδα anti-bounce (reset_synchroniser_antibounce), μία μονάδα leddecoder (leddecoder_inst), αναθέτοντας κάθε bit της εξόδου της στη κατάλληλη κάθοδο (στο dp αναθέτει 1), και μία μονάδα leddriver (leddriver_inst). Μόνο το anti-bounce χρησιμοποιεί το αρχικό ρολόι, ενώ οι υπόλοιπες μονάδες χρησιμοποιούν το αργό ρολόι. Αναθέτει επίσης δοκιμαστικά, την τιμή 5 σε ένα 4bit καλώδιο "char" (που θα αποτελέσει είσοδο στο leddecoder_inst). (Με τον χαρακτήρα θα ασχοληθούμε κυρίως στα επόμενα κομμάτια της εργασίας).

Για να ελεγχουμε σωστά το πρόγραμμα, κατασκευάζουμε testbench για καθεμία από τις υπομονάδες. Για το antibounce αρχικά, μειώνουμε τη τιμή του counter στη μονάδα από 32-bit σε 4-bit για να γίνει ο έλεγχος ευκολότερος. Για τις εισόδους, αναθέτουμε τυχαία το κουμπί, με μεγάλες και μικρές καθυστερήσεις μέχρι την απενεργοποίησή του, και βάζουμε το ρολόι να εναλλάσσεται συνεχώς μέσω ενός forever block ανά 5ns έτσι ώστε να έχει περίοδο 100MHz. Για το leddriver και το top το testbench είναι όμοιο, αφού η μόνη είσοδος που μπορούμε να ελεγχουμε είναι και στα δύο το reset. Αναθέτουμε το ρολόι όπως και στο antibounce, ενώ ενεργοποιούμε το reset αρχικά ελάχιστα και έπειτα για μεγάλο χρονικό διάστημα.

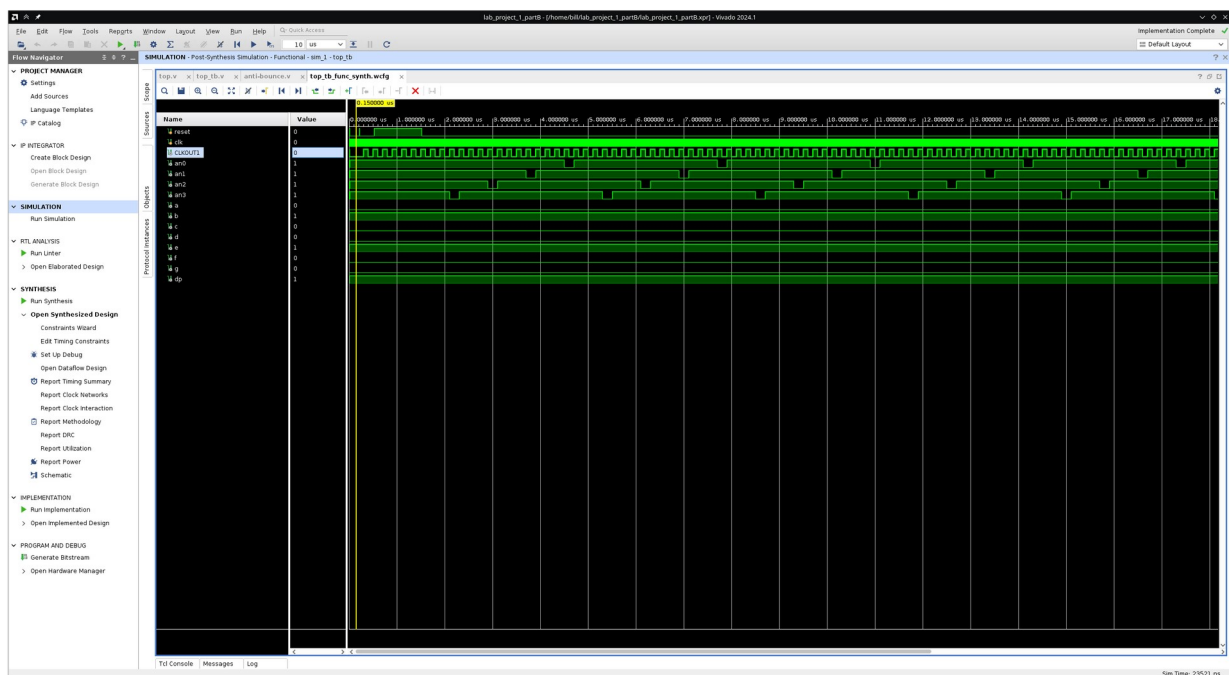
Ο κύριος έλεγχος του κυκλώματος έγινε δια μέσω του vivado. Για τον έλεγχο της σωστής λειτουργίας έγινε έλεγχος με behavioural simulation, καθώς και με post synthesis και post implementation simulation (functional και timing). Για την εκμετάλευση των προσομοιώσεων αυτών, έγινε έλεγχος των κυματομορφών που παρήγαν μεταξύ τους.

Το πλαίσιο δοκιμής αυτό έδειξε έναν αριθμό σχεδιαστικών προβλημάτων, κυρίως στην ανάθεση λανθασμένων τμών στην μονάδα MMCM, όπως και στη λανθασμένη χρήση δεύτερου always block στην leddriver μονάδα, που παρατηρήθηκε λόγω της μη-λειτουργίας του reset.

Παρακάτω εμφανίζονται δύο στιγμιότυπα που εμφανίζουν το post-synthesis και post-implementation functional simulation:



Part B Top Module Post-Implementation Functional Simulation

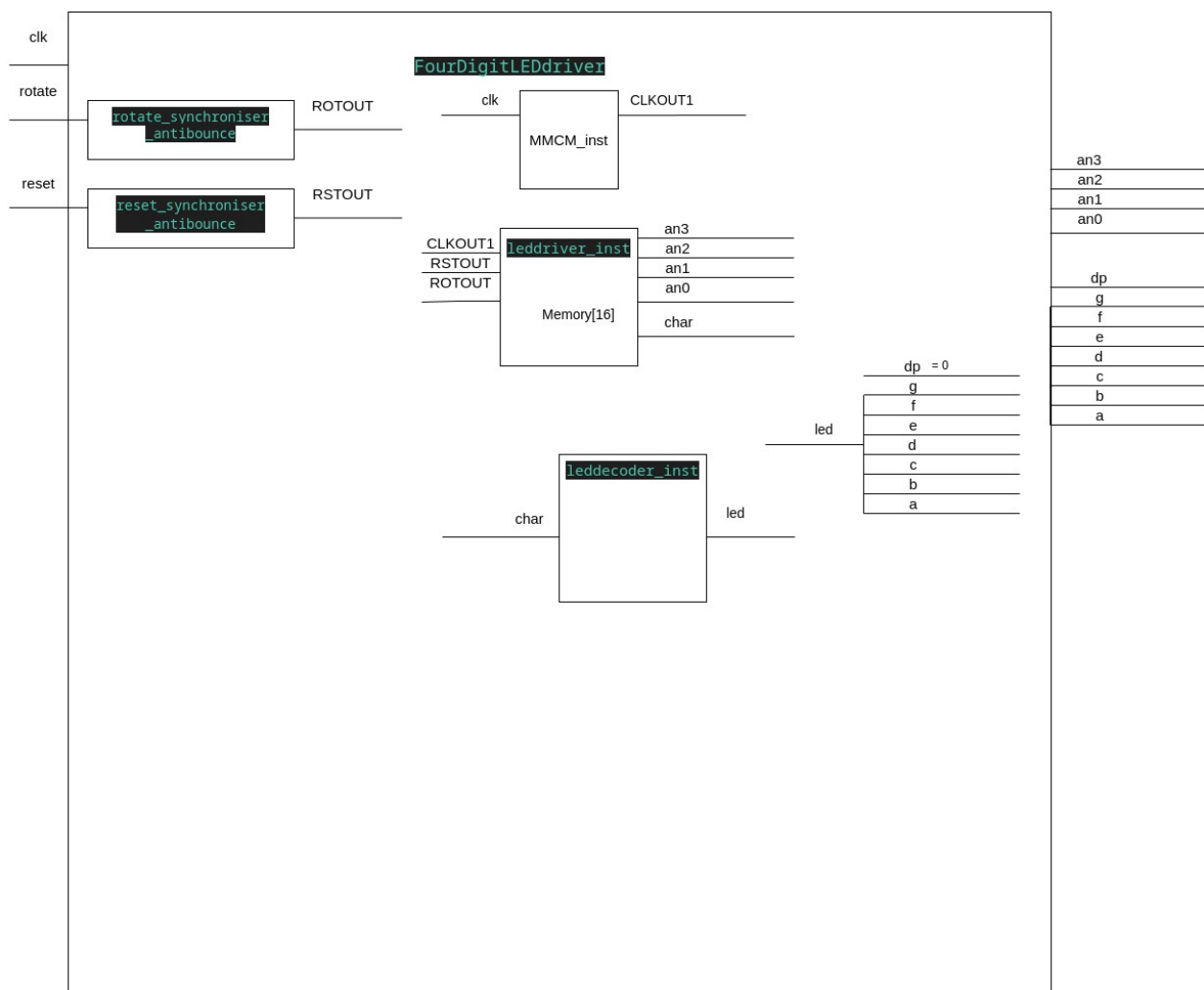


Part B Top Module Post-Synthesis Functional Simulation

Στην FPGA χρειάστηκαν δύο επαναπρογραμματισμοί, καθώς στον πρώτο προγραμματισμό δεν λειτουργούσε το reset, κάτι που οδήγησε στην αλλαγή λογικής στην Iddriver μονάδα που αναφέρθηκε και παραπάνω. Πέρα από αυτό το πρόβλημα όμως δεν παρουσιάστηκε κάποια άλλη δυσκολία, και δέν αντιμετωπίστηκε κάποιο ηλεκτρικό πρόβλημα. Μετά τη διόρθωση του προβλήματος η πλακέτα έδειχνε 5 σε 4 από τις ενδείξεις της, η στιγμιαία τιποτα αν πατούσαμε το reset, συμπεριφορά η οποία ήταν η αναμενόμενη

- **Μέρος 3ο – Βηματική περιστροφή του μηνύματος με χρήση κουμπιού:**

Σε αυτό το κομμάτι της εργασίας, θα προσαρμόσουμε το κύκλωμα ώστε να έχουμε ένα μήνυμα (ένα σύνολο απο χαρακτήρες) σε έναν καταχωρητή, “περιστρέφοντάς το” με τη χρήση ενός κουμπιού (συγκεκριμένα του κουμπιού BTNR). Συγκεκριμένα θα επεκτείνουμε τη λειτουργία της leddriver μονάδας ώστε να έχει τον καταχωρητή με το μήνυμα και να αλλάζει την τιμή του χαρακτήρα που θα οδηγήσει τις καθόδους, ανάλογα με το πόσες φορές πατήθηκε το κουμπί καθώς και ανάλογα με τις τιμές των ανόδων. Αξίζει να σημειωθεί ότι θα επεκτείνουμε τα constraints έτσι ώστε πλέον να αναγνωρίζεται το κουμπί BTNR (που αντιστοιχεί στην ακίδα M17) σαν είσοδος. Ένα διάγραμμα για την ροή των δεδομένων σε αυτό το κομμάτι φαίνεται παρακάτω:



PartC Dataflow Diagramm

Για να το πετύχουμε αυτό προσθέτουμε στην leddriver μονάδα το κουμπί σαν είσοδο, έναν χαρακτήρα σαν έξοδο, και ένα δεύτερο always block (με ασύγχρονο reset), το οποίο θα είναι υπεύθυνο για να υποδυκνύει ποιο κομμάτι του μηνύματος θα πρέπει να φαίνεται στη πλακέτα. Αρχικά θα χρειαστεί να κρατήσουμε τέσσερις θέσεις του καταχωρητή την μία δίπλα απο την άλλη, κάτι που επιτυγχάνεται με τη χρήση τεσσάρων μεταβλητών. Επίσης θα πρέπει να αλλάζουμε τις θέσεις αυτές κατα 1 κάθε φορά που θα χρησιμοποιείται το κουμπί. Θα κρατήσουμε επίσης την προηγούμενη τιμή του κουμπιού έτσι ώστε να μην αλλάζει συνεχώς το μήνυμα αν το κουμπί μείνει πατημένο. Συγκεκριμένα, όταν η προηγούμενη κατάσταση του κουμπιού είναι “μή πατημένο” αλλά το κουμπί πατιέται, τότε καθεμία απο τις επιμέρους μεταβλητές που δείχνουν τη θέση του

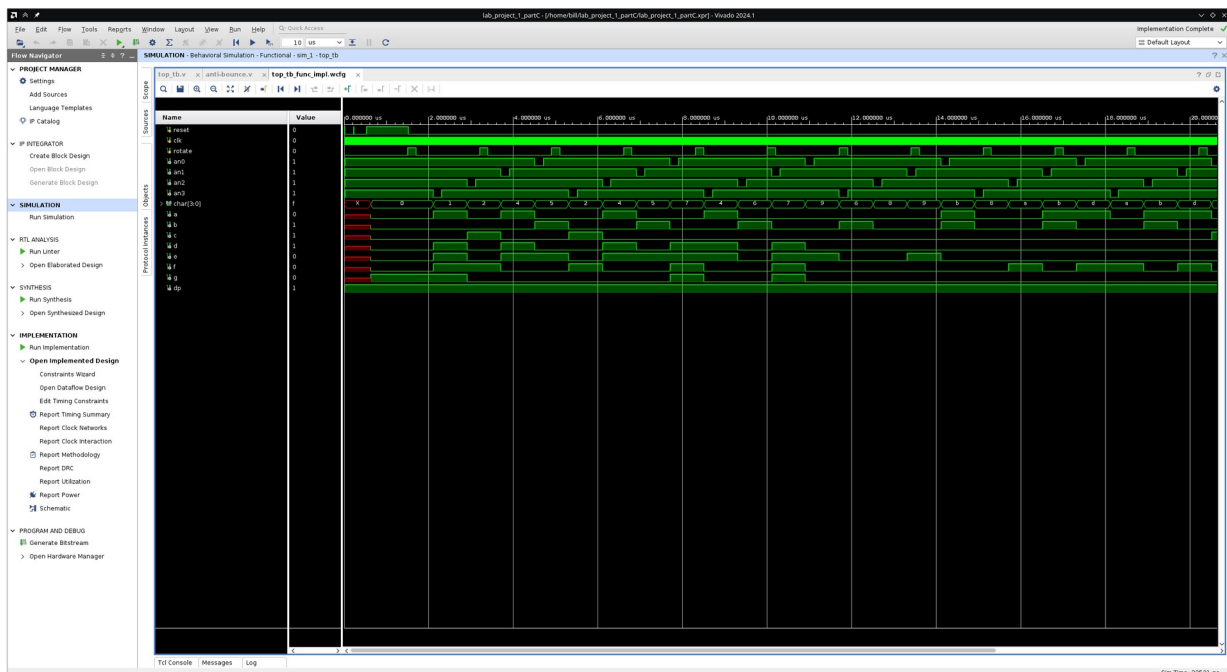
μυνήματος θα αυξηθούν κατά ένα. Στο reset, θα αρχικοποιήσουμε το μήνυμα και θα αναθέσουμε στις μεταβλητές τις πρώτες θέσεις του μυνήματος

Θα χρειαστεί επίσης να προσαρμόσουμε ελαφρώς και την πρώτη always, ώστε να αναθέτουμε στην έξοδο τη σωστή θέση του μυνήματος, ανάλογα με την άνοδο που είναι ανοιχτή. Για παράδειγμα αν η πρώτη άνοδος είναι ανοιχτή, θα θέσουμε τον χαρακτήρα που θα έχουμε σαν έξοδο τη θέση του μυνήματος που υποδυνάμει η πρώτη μεταβλητή θέσης, αν η δεύτερη είναι ανοιχτή θα θέσουμε τη θέση που υποδυνάμει η δεύτερη μεταβλητή κ.α.

Θα χρειαστεί να αλλάξουμε ελαφρώς και την top μονάδα. Θα συνδέσουμε πρώτα το νέο κουμπί με μια δεύτερη μονάδα anti-bounce. Επίσης θα βγάλουμε την ανάθεση που κάνουμε στον χαρακτήρα, καθώς για αυτό είναι πλέον υπεύθυνη η leddriver μονάδα.

Θα χρειαστεί να προσαρμόσουμε ελαφρώς τις μονάδες δοκιμών της leddriver και της top μονάδας. Συγκεκριμένα, και στα δύο testbench θα προσθέσουμε ένα forever block μέσα στο οποίο θα ενεργοποιούμε το κουμπί ανα 1500ns και θα το απενεργοποιούμε μετά από 200ns. Έτσι θα μπορούμε να ελεγχουμε και τις καινούριες προσαρμογές που έγιναν στο πρόγραμμα. Θα αλλάξουμε επίσης ξανά τον μετρητή στο anti-bounce με μία μικρότερη τιμή για διευκόλυνση της δοκιμής. Όπως και παραπάνω, θα δοκιμάσουμε τις μονάδες με behavioral, post-synthesis και post-implementation simulation και θα συγκρίνουμε τις κυματομορφές των προσομοιώσεων.

Δεν προέκυψε κάποια τεχνική δυσκολία κατά τη δοκιμή του κυκλώματος στο testbench. Παρακάτω φαίνεται ένα simulation στο οποίο φαίνεται η σωστή επιστροφή του μυνήματος από το κύκλωμα. Τα post-synthesis και post-implementation simulations δεν απέκλιναν από το behavioural simulation αξιοσημείωτα οπότε παραλείπονται



Part C Top Module Behavioural Simulation

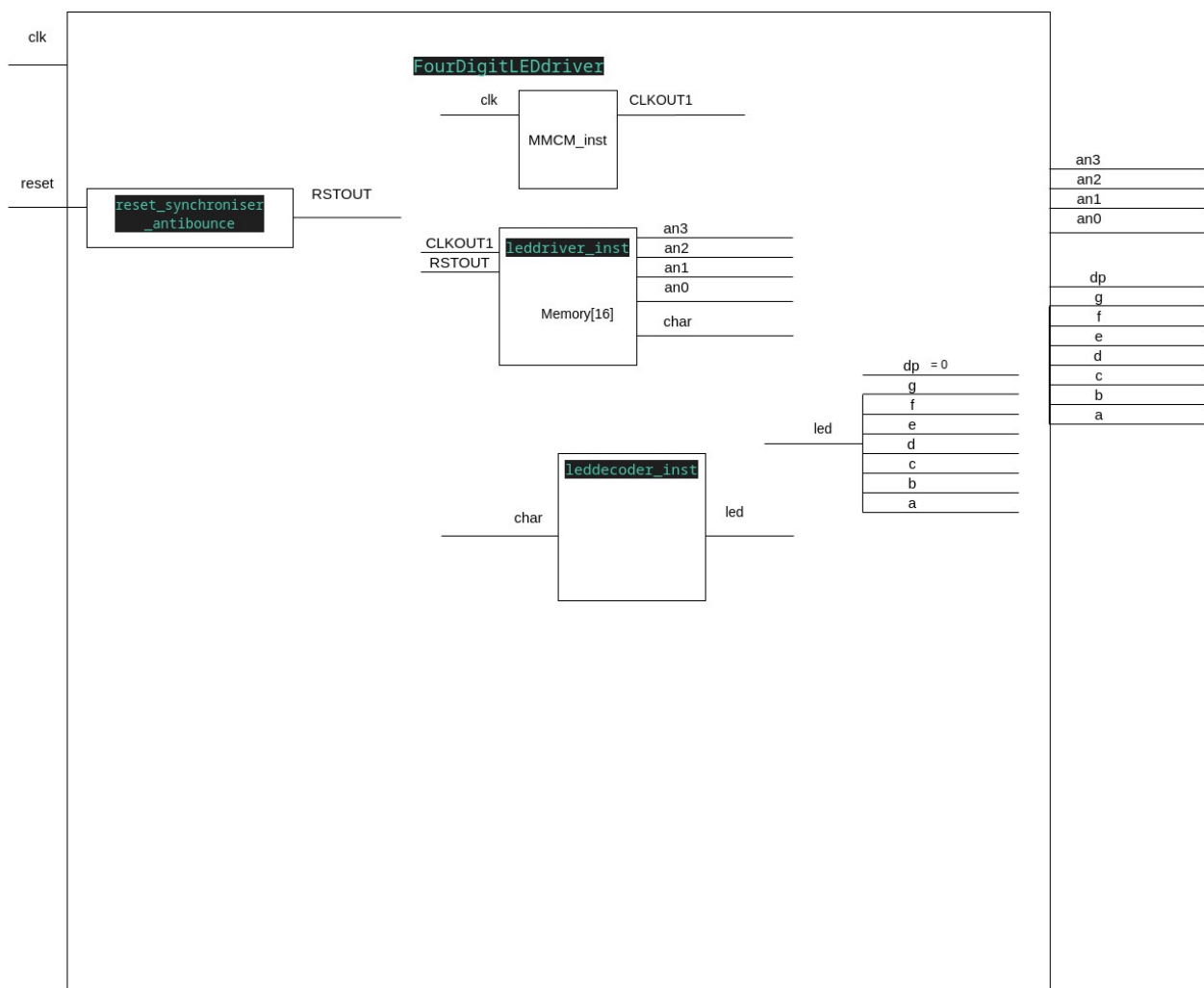
Για τη σωστή λειτουργία στη πλακέτα, χρειάστηκαν δύο επαναλήψεις, αυτό διότι παρέλειψα να επαναφέρω αρχικά τον μεγαλύτερο μετρητή της anti-bounce μονάδας που άλλαξα λόγω διευκόλυνσης για το testbench. Αυτή όμως υπήρξε η μοναδική δυσκολία που αντιμετώπισα στον προγραμματισμό της, καθώς το κύκλωμα είχε την αναμενόμενη λειτουργία.

- **Μέρος 4ο - Βηματική περιστροφή του μυνήματος με σταθερή καθυστέρηση:**

Στο τελικό κομμάτι της εργασίας, θα προσαρμόσουμε ξανά το κύκλωμα έτσι ώστε η περιστροφή του μυνήματος να μην εξαρτάται από το κουμπί αλλά από μια σταθερή καθυστέρηση. Αλλάζουμε ξανά τη μονάδα leddriver, συγκεκριμένα το δεύτερο always block, έτσι ώστε η περιστροφή να εξαρτάται από έναν μετρητή 23-bit που μετράει 1,6777214 δευτερόλεπτα (το οποίο επιτυγχάνεται χρησιμοποιώντας το αργό ρολόι που χρησιμοποιεί η μονάδα). Συγκεκριμένα ο μετρητής γίνεται 1 στο reset, και αυξάνεται με κάθε κύκλο του ρολογιού. Κάθε φορά που ο μετρητής μηδενίζεται το μήνυμα περιστρέφεται.

Από την top μονάδα και το constraint, θα χρειαστεί απλώς να αφαιρέσουμε το κομμάτι της λογικής που αφορά το κουμπί. Δηλαδή από το constraint θα αφαιρέσουμε την είσοδο που αφορά το κουμπί, και από το top θα αφαιρέσουμε τη μονάδα anti-bounce που συνδέαμε στο κουμπί.

Ένα διάγραμμα για την ροή των δεδομένων σε αυτό το κομμάτι φαίνεται παρακάτω:

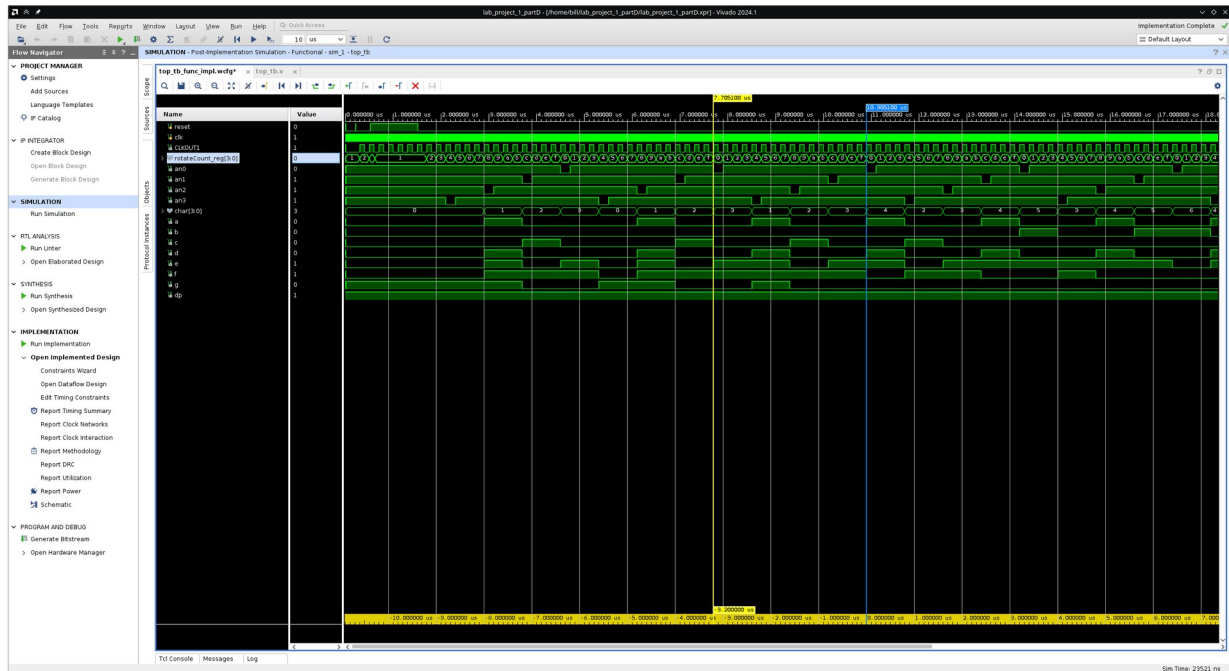


PartD Dataflow Diagramm

Για τις μονάδες ελέγχου αρκεί να αφαιρέσουμε τα κομμάτια που αφορούσαν το κουμπί περιστροφής από το top testbench και το leddriver testbench. Τα διανύσματα ελέγχου δηλαδή θα είναι όμοια με αυτά που χρησιμοποιήσαμε για τη δοκιμή του δεύτερου κομματιού της εργασίας. Θα μικρύνουμε επίσης πάλι, τον μετρητή της μονάδας anti-bounce, αλλά αυτή τη φορά και τον μετρητή που προσθέσαμε στην leddriver μονάδα σε αυτό το κομμάτι της εργασίας, για τον ίδιο

λόγο (συγκεκριμένα, θα κάνουμε και τους δύο 4-bit). Όπως και παραπάνω, θα δοκιμάσουμε τις μονάδες με behavioral, post-synthesis και post-implementation simulation και θα συγκρίνουμε τις κυματομορφές των προσομοιώσεων.

Παρακάτω φαίνεται το post-implementation simulation στο οποίο φαίνεται η σωστή επιστροφή του μυνήματος απο το κύκλωμα. Έχουν προστεθεί δύο κέρσορες σε σημεία όπου ο μετρητής μηδενίζεται, δηλαδή το μήνυμα προχωράει. Αξίζει να σημειωθεί ότι δεν αλλάζουν τιμή οι καταχωρητές-δείκτες μετά τον πρώτο μηδενισμό του μετρητή, δηλαδή χάνεται μία περιστροφή μετά το reset.



Part D Top Module Post-Implementation Simulation

Η πλακέτα χρειάστηκε να προγραμματιστεί μόνο μια φορά. Η συμπεριφορά που αναφέρθηκε νωρίτερα όμως, δηλαδή η μή περιστροφή του μυνήματος μετά τον πρώτο μηδενισμό του μετρητή παρατηρήθηκε και στη πλακέτα. Πέρα του προβλήματος αυτού, η δοκιμή διεξάχθηκε ομαλά.

- **Συμπεράσματα:**

Η πορεία της εργασίας ήταν αρχικά ελαφρώς απότομη, κυρίως επειδή χρειαζόταν καινούριες γνώσεις, μετά την ολοκλήρωση του δεύτερου κομματιού όμως η εργασία συνέχισε ομαλά. Όσον αφορά τη σχεδίαση, η προσαρμογή του κυκλώματος λαμβάνοντας υπόψη την συνθεσιμότητα ήταν κάτι καινούριο, οπότε χρειάστηκε να προσαρμώσω τη λογική μου κατάλληλα, κάτι που κατάφερα σε έναν ικανοποιητικό βαθμό για την εργασία με την ολοκλήρωση του δεύτερου μέρους. Για την κατάλληλη επαλήθευση του κυκλώματος η μεγαλύτερη δυσκολία ήταν η εκμάθηση ενός καινούριου εργαλείου, πρόβλημα που επίσης έπρεπε να λυθεί πριν την ολοκλήρωση του δεύτερου μέρους. Στην δοκιμή αντιθέτως δεν προέκυψαν αξιοσημείωτα προβλήματα, και αυτά που εμφανίστηκαν μπορούσαν να παρατηρηθούν με πιο προσεκτικό έλεγχο κατά το στάδιο της επαλήθευσης.